FRAMEWORKS PARA ESCALAR SCRUM – NEXUS

Dado que Scrum impone un límite en la cantidad máxima de integrantes del Equipo de Desarrollo, surge la necesidad de escalar este Framework para poder gestionar productos de mayor envergadura, ya que las practicas agiles se utilizan en ambientes complejos para reducir tal complejidad. (Nexus, scale scrum, less, SAFe.)

Framework Nexus

- Framework que consiste en roles, eventos, artefactos y técnicas que vinculan el trabajo de aproximadamente tres a nueve equipos Scrum que trabajan sobre un único Product Backlog común para la construcción de un incremento integrado de producto “Terminado”.
- Con un solo Product Owner para entregar un único producto.
- Surge para lidiar con la complejidad que supone varios Equipos Scrum trabajando sobre un mismo Product Backlog. Complejidad dada por:
    - Requerimientos: el alcance de estos puede superponerse. La forma en que se implementan también puede afectar a los demás.
    - Conocimiento del dominio: el conocimiento del sistema de negocio debería mapearse a los Equipos Scrum para minimizar las interrupciones entre los mismos durante el Sprint.

Responsabilidades

- Nexus Integration Team: Responsable de asegurar que se produzca un incremento integrado al menos una vez en cada sprint. La integración incluye abordar las restricciones técnicas y no técnicas del equipo multifuncional que pueden impedir la capacidad de un Nexus para entregar constantemente un incremento integrado.
- Product Owner: Es responsable de maximizar el valor del producto y el trabajo realizado e integrado por los Scrum Teams en un Nexus, así mismo también es responsable de la gestión eficaz del Product Backlog.
- Scrum Master: responsable de asegurar que el marco de trabajo Nexus se entienda y se promulgue como se describe en la Guía de Nexus. Puede ser un Scrum Master en uno o más de los scrums teams en el Nexus.
- Uno o más miembros del Nexus Integration Team: a menudo está formado por miembros de los Scrum Teams que ayudan a los Scrum Teams a adoptar herramientas y prácticas que contribuyen a mejorar la capacidad de los Scrum Teams para entregar un Integrated Increment útil y de valor que cumple con la Definición de Terminado.

Eventos

Nexus agrega o extiende los eventos definidos por Scrum. La duración de los eventos Nexus se guía por la duración de los eventos correspondientes en la Guía de Scrum. Tienen definido un bloque de tiempo adicional a sus correspondientes eventos de Scrum.

- Nexus Sprint: Los scrums teams producen un solo Integrated Increment (Incremento de integración), es lo mismo que scrum.
- Refinamiento entre equipos: El Refinamiento Entre Equipos del trabajo del Product Backlog reduce o elimina las dependencias entre equipos dentro de un Nexus. El Product Backlog debe descomponerse para que las dependencias sean transparentes, se identifiquen entre equipos y se eliminen o se minimicen. El

Refinamiento Entre Equipos del Product Backlog a escala sirve a un propósito dual:
    - Ayuda a los Scrum Teams a prever qué equipo entregará qué elementos del Product Backlog.
    - Identifica dependencias entre estos equipos.

El Refinamiento Entre Equipos es continuo. La frecuencia, duración y participación del Refinamiento Entre Equipos varía para optimizar estos dos propósitos.

- Nexus Sprint Planning: El propósito de la Nexus Sprint Planning es coordinar las actividades de todos los Scrum Teams dentro de un Nexus para un solo Sprint. Los representantes apropiados de cada Scrum Team y el Product Owner se reúnen para planificar el Sprint.
    - Resultados: Objetivo de Sprint para cada Scrum team.- Objetivo de Nexus. - Un Nexus sprint backlog. - Un Sprint backlog.
- Nexus Daily Scrum:
    - Se discuten problemas de integración y se inspecciona el progreso hacia el objetivo de sprint del nexus.
    - Solo asisten representantes de cada Scrum team.
    - La Daily Scrum de cada Scrum Team complementa la Nexus Daily Scrum creando planes para el día, centrados principalmente en abordar los problemas de integración planteados durante la Nexus Daily Scrum.
    - Los Scrum teams no solo se reúnen o coordinan en el Nexus daily scrum, sino que durante el día, puede existir comunicación entre equipos con mayor detalle sobre adaptar o replanificar el resto del trabajo del sprint.
- Nexus Sprint Review: se lleva a cabo al final del Sprint para brindar retroalimentación al Integrated Increment terminado que el Nexus ha construido durante el Sprint y determinar adaptaciones futuras. Reemplaza a las sprint reviews de cada scrum team.
- Nexus Sprint Retrospective: El propósito es planificar formas de mejorar la calidad y la eficacia en todo el Nexus. El Nexus inspecciona cómo fue el último Sprint con respecto a individuos, equipos, interacciones, procesos, herramientas y su Definición de Terminado. Además de las mejoras de cada equipo, las Retrospectivas de los Scrum Teams complementan la Nexus Sprint Retrospective mediante el uso de inteligencia de abajo hacia arriba para centrarse en los problemas que afectan al Nexus en su conjunto.

Artefactos

Cada artefacto contiene un compromiso, y estos existen para reforzar el empirismo y el valor de Scrum para el Nexus y sus interesados.

- Product Backlog: Hay uno solo que contiene la lista de todo lo que el Nexus y sus Scrum Teams necesitan para mejorar el producto. El Product Backlog debe entenderse con tal nivel que permita detectar y minimizar las dependencias. Es responsabilidad del Product Owner.
    - El compromiso es el Objetivo del Producto, que describe el estado futuro del producto y sirve como un objetivo a largo plazo para el Nexus.
- Nexus Sprint Backlog: está compuesto del Objetivo del Sprint de Nexus y elementos del Product Backlog de cada Scrum Team del Nexus. Se usa para resaltar las dependencias y el flujo de trabajo durante el Sprint, y se actualiza durante el mismo a medida que se obtiene más conocimiento. Este debe tener un

nivel de detalle tal que permita que Nexus pueda inspeccionar su progreso en la Nexus Daily Scrum.
    - Su compromiso es el Objetivo de Sprint de Nexus, que es un único objetivo para Nexus. Es la suma de todo el trabajo y los Objetivos de Sprint de los Scrum Teams dentro del Nexus. Se crea en la Nexus Sprint Planning y se agrega al Sprint Backlog de Nexus.
- Integrated Increment: Es la suma actual de todo el trabajo integrado completado por un Nexus en relación con el Objetivo del Producto. Se inspecciona en la Nexus Sprint Review pero puede entregarse antes de la misma. Debe cumplir con la definición de terminado.
    - El compromiso de este artefacto es la Definición de Terminado, que define el estado del trabajo integrado cuando cumple con la calidad y las medidas requeridas para el producto. El incremento se termina sólo cuando está integrado, es de valor y utilizable. Es responsabilidad del Nexus Integration team una definición de terminado que se pueda aplicar en cada sprint, y todos los Scrum Team deben definir y adherirse a esta definición

LEAN Y KANBAN

Mejora continua de procesos con Kanban

- Es una filosofía de gestión que surge en la manufactura, en la línea de producción, no en el mundo del software. En particular surge en Toyota, donde se empieza a trabajar y a relacionarse con conceptos de Optimidad.
- La filosofía “Lean”, conduce a una visión integrada de la cultura y la estrategia para atender al cliente final con alta calidad, bajo costo y tiempo de entrega, produciendo exactamente lo que el cliente final quiere, cuando lo quiere, dónde lo quiere, a un costo mínimo y precio justo.
- Utiliza el método Just in Time, el cual permite el intercambio del producto entre fases del proceso de desarrollo o a la entrega del producto final al cliente justo a tiempo.
- Principios (Estos principios apuntan al desarrollo de productos en la industria de manufactura.):
    - Eliminar desperdicios (quizás el más importante): Debemos tener un proceso que sea eficiente y que sume valor en todos los pasos del proceso, todo paso o parte que no contribuye a la generación de valor produce un desperdicio.
        - Lean apunta a minimizar el re trabajo y no hacer actividades que no agreguen valor al negocio.
        - Surge a partir del concepto de Just in Time.
        - Desperdicios en SW: Características extra, trabajo a medias, proceso extra, movimiento, defectos, esperas, cambio de tareas.
    - Amplificar el aprendizaje: Crear y mantener una cultura de mejoramiento continuo y solución de problemas.
        - Se basa en poder aprender a partir del trabajo que estamos haciendo y que ese conocimiento sea accesible a toda la organización.
        - Tienen que ver con el valor que le damos a la figura del equipo y a la autogestión del mismo, esto es gracias a la amplificación del aprendizaje y a la mejora continua a través del mismo.
        - Crear mecanismos (como revisiones, reuniones, documentación, tests) para que el aprendizaje de una persona o un equipo se extienda y beneficie a toda la organización.

- Embeber la integridad conceptual: Encastrar todas las partes del producto o servicio, que tenga coherencia y consistencia (tiene que ver con los requerimientos no funcionales). La integración entre las personas hace el producto más integro.
    - La calidad del producto no se negocia y la simplicidad es un factor clave.
    - El producto tiene que satisfacer las expectativas y necesidades del cliente, esa es la principal prioridad y cada decisión se alinea con esto.
    - Integridad Conceptual: todos los componentes del sistema trabajan en forma coherente en conjunto.
    - Integridad Percibida: el producto total tiene un balance entre función, uso, confiabilidad y economía que le gusta a la gente.
- Diferir compromisos hasta el último momento responsable: Tomar decisiones al último momento en el cual es necesario hacerlo. Relacionado con el principio ágil “Decidir lo mas tarde posible pero responsablemente”
    - Tenemos que tomar la decisión en el último momento en donde nosotros nos aseguremos que no perdemos nada por no tomar la decisión (último momento responsable).
    - Si nos anticipamos vamos a tener información parcial.
- Dar poder al equipo: Se busca respetar a la gente: entrenar líderes, fomentar buena ética laboral y delegar decisiones y responsabilidades del producto en desarrollo al nivel más bajo posible.
- Ver el todo: Tenes una visión holística, de conjunto (el producto, el valor agregado que hay detrás, el servicio que tiene los productos como complemento)
    - La idea es que los objetivos particulares de las partes estén alineados con los objetivos globales.
- Entregar lo antes posible: Estabilizar ambientes de trabajo a su capacidad mas eficiente y acotar los ciclos de desarrollo.
    - Entregar rápido implica estabilizar ambientes de trabajo a su capacidad más eficiente y acotar los ciclos de desarrollo.
    - Entregar rápidamente hace que se vayan transformando “n” veces en cada iteración. Incrementos pequeños de valor;

KANBAN (Framework)

- Es un framework/método (no metodología) cuyo propósito es mejorar / introducir cambios en mis procesos para ir evolucionando permanentemente.
- Busca basarse en los principios Lean y teniendo en mente los conceptos de “principios KANBAN” generar un espacio de trabajo que obtenga un producto de calidad.
    - Fomenta la evolución gradual de los procesos existentes.
    - No pide una revolución, si no que fomenta el cambio gradual.
    - Esta basado en una idea muy simple: Limitar e ltrabajo en progreso (WIP)
    - El Kanban (tarjeta de señal) implica una señal visual que se produce para indicar que el nuevo trabajo se puede tirar (“pull”) porque el trabajo actual no es igual la limite acordado.
- Es el referente principal de Lean en software.
- Sirve para gestionar cosas para las cuales Scrum no es muy útil.

- Es un enfoque que se usa para la gestión del cambio, no es un proceso de desarrollo ni una metodología de administración de proyecto, es un proceso para gestionar el cambio en procesos de desarrollo de software
    - KANBAN (en mayus) → Framework de mejora continua basado en los principios lean.
    - Kanban (minus) → Tarjeta de señal / señal de indicación (se usa en el sistema para visualizar el trabajo)
- Relacionado con el concepto de JUST IN TIME: No hacer cosas que no son necesarias ahora o no me han pedido porque pueden cambiar.
- Trabaja con la teoría de administración de colas. Por ende necesitamos tener mapeado el proceso a un conjunto de colas de trabajo.
    - El pedido o solicutid va a ir pasando por cada una de las colas de trabajo donde cada una le aportara al producto/servicio final que se entregara.

Principios

- Visualizar el flujo:
    - Hacer el trabajo visible constantemente.
    - Kanban se basa en el uso de tableros visuales para representar el flujo de trabajo.
- Limitar el trabajo en progreso (Work In Progress)
    - promueve la idea de limitar la cantidad de trabajo en curso en cualquier momento dado. Esto evita la sobrecarga del equipo y ayuda a mantener un flujo de trabajo equilibrado y constante.
- Gestionar el flujo
    - Busca lograr un flujo de trabajo continuo y eficiente.
    - Se deben identificar y resolver los cuellos de botella y los problemas que afectan el flujo de trabajo
- Hacer explicitas las políticas.
    - requiere que las políticas de trabajo y los procedimientos estén explícitos y sean claros para todos los miembros del equipo.
- Establecer ciclos de retroalimentación
    - Parte esencial para cualquier proceso que nos permite realizar cambios evolutivos.
    - Debemos definir con qué frecuencia realizar las reuniones.
    - Realizar revisiones demasiado frecuentes puede generar cambios constantes sin haber evaluado completamente el impacto de los ajustes anteriores. Sin embargo, si las revisiones se realizan con poca frecuencia, los problemas o ineficiencias pueden mantenerse durante mucho tiempo, afectando el rendimiento del proceso (hay que buscar cadencia adecuada).
    - Fomenta colaboración, aprendizaje y mejora.
    - Se basa en los datos.
- Mejorar de forma colaborativa y evolutiva.
    - Fomenta la mejora continua a través de la colaboración y el aprendizaje.

Como aplicar Kanban

- Primero, entendiendo lo que tenemos hoy, como trabaja el equipo y que hace, poder graficarlo en un tablero.
- Luego, acuerdo los límites del trabajo en progreso (WIP), en cada una de las etapas voy a definir la cantidad máxima de tarjetas que puedo tener en progreso en cada una de las etapas, que va a estar vinculada directamente con la cantidad de recursos que yo tenga trabajando en esa parte del proceso.
- Pasos:

- Empezar con lo que se tiene ahora.
- Entender el proceso actual.
- Acordar los limites de WIP para cada etapa del proceso.
- A continuación, comienza a fluir el trabajo a través del sistema tirando de él, en presencia de señales Kanban

- Dividir el trabajo en piezas:
  - Pueden ser US, requerimientos, incidentes, etc.
  - Luego de dividir el trabajo, podemos tipificar las piezas (tarjetas) en función de lo que estemos modelando.
- Visualizar el flujo de trabajo:
  - Utilizar nombres en las columnas para ilustrar donde esta cada ítem en el flujo de trabajo, cada una de las piezas definidas en el punto anterior van a fluir de izquierda a derecha en las columnas.
  - Uno de los conceptos clave de Kanban es que cuando tengamos estas piezas sean los recursos los que se autoasignan el trabajo y no que alguien asigne el trabajo a un recurso. (funciona como pull no como push)
- Limitar el work in progress (WIP):
  - Asignar límites explícitos de cuántos ítems puede haber en progreso en cada estad del flujo de trabajo, para asegurar que el trabajo fluya.
  - Esto ayuda a prevenir la sobrecarga y a mantener un flujo de trabajo equilibrado.
  - La autoasignación de tareas se ve reflejada con un avatar personalizado.

Resumen de aplicación:



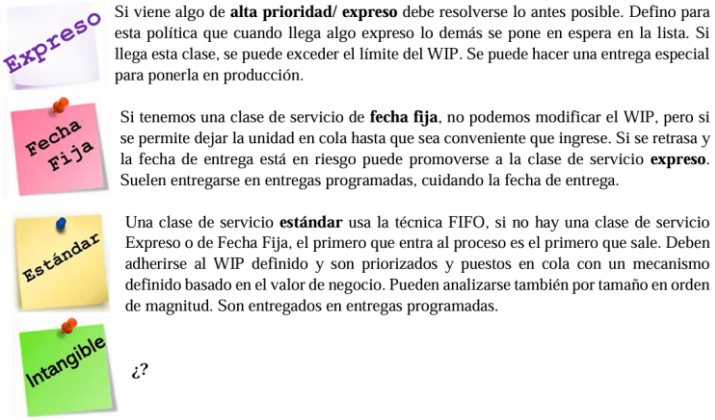


- Definir el proceso, siguiendo el concepto de que Kanban es pull y no push.
  - Cada integrante puede tomar alguna de las piezas que estén en análisis como Hecho y pasarla a En Progreso dentro del desarrollo.
- Definimos unidades de trabajo
  - Debemos dividir el trabajo en piezas, definiendo tipos de trabajo
  - Estas piezas van pasando por distintas colas del proceso (columnas)
  - Usamos colores y notas diferentes para cada tipo de trabajo
  - Se asigna capacidad en función de la demanda (asigno cuanto esfuerzo se le va a dedicar a cada uno según la demanda -por eso vemos porcentajes- y en base a eso defino la capacidad de trabajar)
  - Tipos de trabajo
- Definimos el WIP
  - Si en desarrollo tengo un cuello de botella, es decir, llegue al WIP 4, el trabajo no fluye, por lo tanto, agrego más recursos de otra área para que lo que esta en desarrollo pueda fluir.



- Hacer explícitas las políticas
  - Defino los criterios que deben cumplir las tareas para moverse de una columna a la otra (para evitar acumulación de trabajo defectuoso y asegurando la calidad)
  - Algunas políticas son:
    - Reposición de trabajo en el tablero (cuando, cuanta cantidad, quien)

        - Definir cuando una actividad está terminada y ese elemento de trabajo puede continuar el flujo (criterios de tracción)
        - Limitar el trabajo en curso
        - Políticas de gestión de elementos de trabajo de diferentes clases de servicio
        - Cadencia de las reuniones
- Administrar cuellos de botella y flujo
    - Si tengo un cuello de botella en desarrollo, el trabajo no fluye.
    - Para resolverlo puedo agregar más recursos de otra área o reajusto el WIP para que ese trabajo pueda fluir.
- Definimos clases de servicio
    - Sirven para definir cómo voy a tratar las piezas de trabajo. No es una cola FIFO. Básicamente ayudan a priorizar ciertas unidades de trabajo por sobre otras



- Cadencia
    - Establecer cadencia para las actividades clave del proyecto (releases, planificaciones y revisiones)
    - Programamos y realizamos regularmente un calendario definido para mantener un ritmo constante y una mejor planificación y seguimiento del proyecto

METRICAS EN LOS 3 ENFOQUES DE GESTION

- Cuando hablamos de los componentes de un proyecto de software, uno de los aspectos mencionados era el de monitoreo y control, es importante poder hacerlo para detectar los desvíos y así corregirlos ya que sabemos que los proyectos se atrasaban un día a la vez.
- Una métrica es un número, tiene que ser un valor cuantitativo/objetivo.
- Métricas básicas para un proyecto de SW (Básico en términos de no tengo capacidad de medición en el org y necesito comenzar, por donde arranco? Que?)
    - Tamaño del producto. (Lo que más se uso fue cuantos requerimientos. Ej cant casos de uso X complejidad)
    - Esfuerzo. (Horas personas lineales, una sola persona trabajando, haciendo una tarea por vez. Cuanto?)
    - Tiempo (calendario. Cuando?)
    - Defectos. (defectos del producto. Esta bueno contarlos por severidad, no sirve contar por ej defectos cosméticos)

Las métricas de proyecto están muy relacionadas a la triple restricción.

Métricas de software en TRADICIONAL

Independientemente que este espectro de métricas en el enfoque tradicional sea bastante amplio, podemos subdividirlos en 3 conjuntos que se denominan dominios de métricas. Por dominio de métricas nos referimos al ámbito al que las métricas hacen referencia, al ámbito en el que las métricas sirven o son útiles. Esto de hablar de dominio, tiene sentido solo en los procesos tradicionales/definidos.

Métricas de proceso:

- Son métricas estratégicas a nivel organizacional, orientadas a mejorar el proceso y tienen como objetivo generar indicadores que permitan mejorar los procesos de software a largo plazo.
- Son públicas y se despersonifican las mediciones de personas, áreas o proyectos particulares, etc. Para que sea sobre el proceso de la organización en general, como un todo.
- Son responsabilidad del Ingeniero de Procesos.

Métricas de proyecto:

- Están enfocadas a los recursos que se dedican al proyecto, como costos, esfuerzos, estimaciones y tiempo.
- Son responsabilidad del Líder de Proyecto y permiten al equipo adaptar el desarrollo de los proyectos y de las actividades técnicas.
- Son privadas de ese proyecto y solo son visibles para los involucrados en el mismo.
- Se utilizan para mejorar la planificación del desarrollo, generando ajustes que eviten retrasos, reduzcan riesgos potenciales y por lo tanto, problemas.
- Sirven para evaluar la calidad de los productos en todo momento.

Métricas de Producto:

- Están enfocadas en lo que se construye, son responsabilidad del equipo de desarrollo y de Testing y son particulares de ese producto.
- Se utilizan con propósitos técnicos y tienen como objetivo generar indicadores en tiempo real de la eficacia del análisis, el diseño, la estructura del código, la efectividad de los casos de prueba y calidad del software a construir.

Métricas de software en AGIL

- Las métricas sirven para un equipo y no son extrapolables por lo que dice el agilismo a otros proyectos o equipos. En ágil hay un principio que habla específicamente de métricas (la mejor métrica de progreso es el software funcionando).
- Este principio apunta a que vamos a medir producto (no proceso ni proyecto).
- Solo se debe medir lo que sea necesario y nada más, es decir, lo que agregue valor para el cliente.

Velocidad:

- Es la métrica más importante del enfoque (métrica de producto), y mide la cantidad de puntos de historia que se realizaron en un Sprint y fueron aceptados por el Product Owner. Hay que recordar que no se cuentan las historias parcialmente terminadas, sólo las que están completadas y aceptadas por el PO.

- se calcula (no se estima) luego de que el PO. acepte la implementación de una US.

Capacidad:

- Es una estimación de cuanto trabajo (puntos de historia) va a poder completar el equipo en el próximo sprint.
- Por ello, cuando tengo una velocidad estable y tengo en claro los recursos que tengo disponibles, es fácil determinar la capacidad, y en la planning definir que features voy a incluir en el sprint backlog.

Running Tested Features (RTF):

- Cuantos US tengo testeados y funcionando. Pero como no tiene en cuenta la complejidad, no termina siendo muy útil ya que puedo tener muchas pero muy simples.

Métricas de software en KANBAN

El foco de medición de Kanban es el proceso, ya que el sistema de trabajo de Kanban es introducir mejoras en un flujo de trabajo continuo, es decir, no existe un proyecto con principio y fin.

Lead Time o Elapsed Time:

- Métrica de vista o perspectiva del cliente. Es la más importante para el cliente. Mide desde el momento en que el cliente me pide algo (entra al Backlog) hasta que yo se lo entrego.
- Se suele medir en días de trabajo.
- Es el ritmo de entrega.

Cycle Time:

- Mide el tiempo desde que el equipo comenzó a trabajar sobre una funcionalidad pedida, hasta que se lo entrega, eliminando el tiempo de espera en el Backlog.
- Por esto se la considera como una vista interna, que sirve al equipo de trabajo y no es tan relevante para el cliente. Es igual al Lead Time menos el tiempo que estuvo en el Backlog.
- Es la medición más mecánica de la capacidad del proceso.
- La llamamos el Ritmo de Terminación.
- Se suele medir en días de trabajo o esfuerzo.

Touch Time:

- Tiempo en el cual un item de trabajo fue realmente trabajado (o tocado) por el equipo. Se refiere a los días hábiles que paso ese item en columnas de “Trabajo en Curso”, en oposición con columnas de cola/buffer y estado bloqueado o sin trabajo del equipo sobre el mismo.

La Eficiencia del Ciclo de Proceso se mide: $$\%\ \text{Eficiencia ciclo proceso} = \frac{Touch\ Time}{Elapsed\ Time}$$

- Mide la eficiencia del tiempo para entregar un trabajo, respecto al tiempo destinado a su ejecución.
- Mientras más cercano a 1 sea su valor, más eficiente es el proceso, ya que todo el tiempo se estuvo trabajando sobre alguna funcionalidad, y no hubo muchos momentos de espera en columnas de acumulación.

Aseguramiento de Calidad de Proceso y de Producto

- Se centra en la implementación de medidas preventivas para garantizar que un producto alcance la máxima calidad posible.
- Esto implica tomar acciones anticipadas y realizar actividades antes de que el producto esté terminado, con el objetivo de asegurar que la calidad esté integrada en todas las etapas del proceso.

¿Qué es la calidad?

- Todos los aspectos y características de un producto o servicio que se relacionan con la habilidad de alcanzar las necesidades manifiestas o implícitas.
- La calidad implica la capacidad de un producto o servicio para satisfacer y superar las expectativas y requisitos del cliente.
- Tiene un problema: está directamente relacionada con la persona (subjetiva).
    - La visión de calidad que tenemos en un momento en el tiempo puede cambiar en otro, ya que la calidad es relativa a las personas.
- Cosas que hacen que un producto no tenga calidad:
    - Atrasos en las entregas, costos excedidos, falta cumplimiento de compromisos, el SW no hace lo que tiene que hacer, trabajo fuera de hora, fenómeno 90-90 (90 hecho y 90 faltante, siempre falta poco).
- Un SW de calidad satisface:
    - Expectativas del cliente, expectativas del usuarios, necesidades de la gerencia, necesidades del equipo de desarrollo y mantenimiento.

Aseguramiento de calidad

- Implica establecer y mantener estándares y procedimientos claros para llevar a cabo las actividades de manera consistente y eficiente.
    - Incluye: definir las mejores prácticas, documentar los procesos, capacitar al personal en la ejecución correcta de las tareas y establecer controles para monitorear y medir el desempeño.
- Además, se centra en garantizar que los productos cumplan con los estándares de calidad establecidos.
    - Implica realizar inspecciones, pruebas y evaluaciones durante el proceso de fabricación para identificar y corregir cualquier defecto o problema antes de que el producto final sea entregado al cliente.

Principios de aseguramiento de calidad

- La calidad no se inyecta ni se compra: La calidad no puede ser agregada o comprada después de la construcción del producto o servicio, debe estar desde el momento 0.
- La calidad conlleva un esfuerzo de todos: no es solo responsabilidad de un departamento o individuo, requiere el compromiso y esfuerzo de todos los miembros del equipo.
- Las personas son clave para lograr la calidad: La capacitación del equipo en calidad es fundamental. El éxito en el desarrollo de software depende en gran medida de las habilidades y conocimientos del personal involucrado.
- Se necesita un sponsor a nivel gerencial: Es importante contar con el respaldo y liderazgo de un patrocinador o sponsor a nivel gerencial que considere el desarrollo de un producto de calidad como algo fundamental y lo respalde con recursos y apoyo.

- Se debe liderar con el ejemplo: Los líderes deben ser ejemplos de compromiso y calidad, demostrando prácticas y comportamientos que fomenten la cultura de aseguramiento de calidad en toda la organización.
- La calidad se mide mediante el testing: Las pruebas son una herramienta fundamental para medir y controlar la calidad del software. No se puede controlar lo que no se mide.
- Simplicidad, empezar por lo básico: En lugar de complicar los procesos, se debe buscar la simplicidad y comenzar por los fundamentos básicos del aseguramiento de calidad.
- El aseguramiento de calidad debe planificarse: Es necesario contar con un plan de aseguramiento de calidad que establezca los objetivos, las actividades y los recursos necesarios para garantizar la calidad del producto o servicio.
- El aumento de las pruebas NO aumenta la calidad automáticamente: aumentar la cantidad de pruebas realizadas no garantiza mayor calidad. Se deben aplicar técnicas y enfoques adecuados para obtener resultados efectivos.
- Debe ser razonable para mi negocio: El aseguramiento de calidad debe adaptarse a las necesidades y características específicas de cada negocio, siendo realista y viable

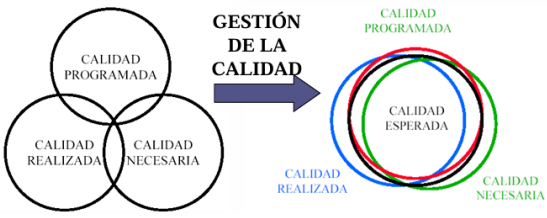
¿Calidad para quien?

- Visión del usuario: esto tiene que ver con las expectativas que tiene el usuario para con el producto, y si este las satisface. Es quizás la más complicada de establecer, porque está en la cabeza de los usuarios, lo cual es una razón más por la que el principio de comunicación constante es crucial, para ir validando en todo momento si estamos en el camino correcto.
- Visión del producto: esto se asocia al nivel de satisfacción de los requerimientos particulares de cada producto.
- Visión del proceso (manufactura): si el proceso de desarrollo utilizado es el correcto para el producto que se desea desarrollar, es decir, el proceso aporta valor al producto y no produce desperdicios.
- Visión del valor: encontrar un equilibrio en la relación costo-beneficio, para obtener siempre el mayor valor para el cliente posible y, obviamente generar ganancias con el desarrollo del software.
- Visión Trascendental: esta visión es un tanto utópica, ya que se asocia a objetivos difíciles de alcanzar. Por ejemplo 0 defectos en el producto, pero son aquellos objetivos que motivan a seguir en busca de la mejora continua.

Si se logra que las 3 perspectivas coincidan disminuimos el riesgo e desperdicio e insatisfacción de los clientes

Calidad programada: Expectativas y estándares de calidad que se establecen para el producto durante su planificación y diseño. (Calidad que se busca alcanzar/objetivo)

Calidad necesaria: Nivel mínimo de calidad requerido para que el producto sea considerado correcto y pueda satisfacer los requerimientos y expectativas del usuario.



Calidad lograda: Calidad real que se ha alcanzado en el producto final una vez completado el proceso de desarrollo y las pruebas correspondientes.

Calidad en el Software

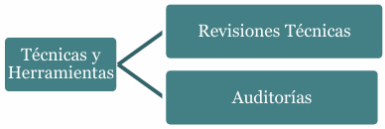
Para desarrollar software de manera efectiva, es fundamental contar con una estructura o guía que oriente al equipo en la construcción del producto deseado (llamada Proceso).

Al crear o adaptar un proceso en una organización, ya sea basado en estándares definidos o en enfoques empíricos, es recomendable basarse en Modelos de Referencia (Modelos para crearlos o Modelos de calidad). Estos modelos proporcionan lineamientos y mejores prácticas para la creación y gestión de procesos, así como para garantizar la calidad del software.

Una vez que se ha establecido un proceso en la organización, incorporando los modelos de referencia pertinentes, es importante buscar una mejora continua del mismo. Para lograr esto, se pueden adoptar Modelos de Mejora de Procesos, los cuales brindan un marco de trabajo para identificar áreas de mejora y establecer proyectos que impulsen dichas mejoras.

Además, es posible formalizar y certificar el cumplimiento de un proceso mediante Modelos de Evaluación, que se utilizan para medir y evaluar el grado de adherencia del proceso a los estándares y modelos de referencia establecidos.

El proceso se materializa en proyectos, los cuales son la unidad de trabajo que da vida a dicho proceso. Al iniciar la ejecución de un proyecto, es esencial incorporar actividades de aseguramiento de calidad, como revisiones técnicas y auditorías, para evaluar la aplicación de los modelos seleccionados.



El proyecto a su vez es el ámbito donde uno trabaja para obtener el producto, la versión de producto que yo someto a evaluación se va generando en el contexto de un proyecto. Para el contexto del producto, existen técnicas y herramientas tanto para el aseguramiento de calidad, como la revisión de pares o las auditorias (De configuración, auditorias físicas y funcionales), como para el control de calidad, como es el Testing.



Cuando yo hago actividades de aseguramiento de calidad tanto de proceso como de producto, se hace en el contexto de un proyecto en específico.



Es importante distinguir el Aseguramiento de calidad de Proceso y el Aseguramiento de calidad de Producto.

	ASEGURAMIENTO DE CALIDAD DE PROCESO	ASEGURAMIENTO DE CALIDAD DE PRODUCTO
FOCO	Se concentra en evaluar y mejorar el proceso de desarrollo de software en sí mismo, es decir, la forma en que las actividades se llevan a cabo.	Se enfoca en evaluar y garantizar la calidad del producto de software resultante, es decir, los artefactos y entregables generados durante el proceso de desarrollo.
OBJETIVO	Busca establecer prácticas efectivas, eficientes y consistentes para el desarrollo de software, con el fin de optimizar la productividad y minimizar los errores o problemas en el proceso	Busca asegurar que el producto de software cumpla con los requisitos establecidos, sea confiable, funcione correctamente y sea adecuado para su uso previsto.
ACT. CLAVE	Involucra definir estándares y buenas prácticas, realizar revisiones técnicas y auditorías de proceso, medir y monitorear indicadores de calidad del proceso, y gestionar la configuración del software.	Incluye realizar pruebas de software (como pruebas funcionales, de rendimiento o de seguridad), realizar revisiones de código, realizar auditorías de producto, verificar el cumplimiento de estándares de calidad y realizar validaciones con los usuarios o stakeholders del producto.

Calidad en procesos definidos vs Calidad en procesos empíricos

El problema de visión entre los procesos definidos y empíricos se debe a diferentes enfoques sobre la relación entre la calidad del proceso y la calidad del producto.

En el caso de los procesos definidos, se argumenta que si el proceso en sí mismo es de alta calidad y se respeta en el contexto del proyecto, entonces el producto resultante también será de alta calidad.

"El proceso tiene que tener calidad y si el proceso tiene calidad y la gente en el contexto del proyecto lo respeta, como consecuencia el producto que se obtenga también va a tener calidad"

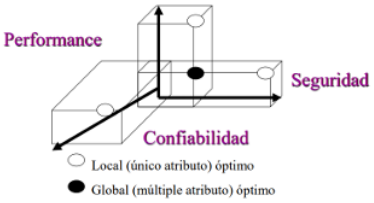
Pero no debe tomarse de manera literal. Aunque un proceso pueda ser de alta calidad, eso no garantiza automáticamente que las personas que trabajan en él cumplirán con todas las tareas y responsabilidades adecuadamente.

Los que trabajan con enfoques empíricos enfatizan que la calidad del producto depende de las personas que participan en el desarrollo. En esta perspectiva, se sostiene que, si el equipo realiza adecuadamente sus tareas y cumple con sus responsabilidades, el producto final tendrá calidad.

"La calidad del producto depende de las personas que participan, si el equipo hace lo que tiene que hacer el producto va a tener calidad"

Calidad en el Producto

- La calidad del producto No se puede sistematizar, no hay modelos en la industria generalizados para evaluar calidad de producto que se puedan aplicar como una plantilla a todos los productos igualmente.
- Su aseguramiento se hace mediante las Revisiones Técnicas (Revisiones de pares) y Auditorias, mientras que su control se hace mediante el Testing.
- Modelos:
    - Modelo de Barbacci: La calidad del producto se mide mediante su confiabilidad, performance y seguridad. Debemos trabajar para lograr un equilibrio entre estas tres cosas.



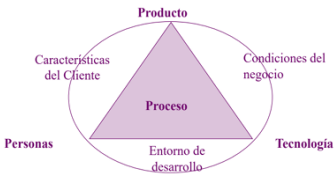
    - Modelo de McCall: La calidad depende de tres factores determinantes que causalmente tiene que ver con las visiones de calidad mencionadas anteriormente.
        - Revisión del producto: Es la capacidad de verificación del producto. Es la facilidad de mantenimiento, flexibilidad y prueba.
        - Operación del producto: Es la calidad del producto en uso, lo que más le importa al usuario final. Corrección, fiabilidad y usabilidad.
        - Transición del producto: Es la facilidad con la que el producto puede expandirse y crecer.



    - NUNCA se puede hacer esa comparación solo contra el modelo, primero se debe definir qué es lo que se supone que el cliente quiere como características de calidad del producto, en términos de requerimientos. La evaluación de calidad sobre un producto se hace en base a que tanto se acerca a esos requerimientos más que lo que se acerca a cumplir con los modelos teóricos.

Calidad en el Proceso

- El proceso es el único factor controlable para mejorar la calidad del Software y el rendimiento como organización.
- El producto, las personas, la tecnología, las características del cliente, las condiciones de negocio y el entorno de desarrollo son incontrolables.



Definición de un proceso de software

- Proceso: Secuencia de pasos ejecutados para un propósito dado (IEEE)
- Proceso de Software: Un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados (Sw-CMM)



- La ingeniería se le llama a la etapa de requerimientos, análisis y diseño. Implementación, es la codificación, el testing y el despliegue. Ahora incorporamos en un proceso de desarrollo de software disciplinas de gestión y de soporte estas son:
    - Planificación y Seguimiento de Proyectos .
    - Administración de Configuraciones.
    - Aseguramiento de la Calidad.

Aseguramiento de calidad de software → Aseguramiento de calidad es insertarse en el proceso de desarrollo de software mientras el software se está construyendo.

- Lo que no está controlado no está hecho.
- Concerniente con asegurar que se alcancen los niveles requeridos de calidad para el producto de software.
- Implica la definición de estándares y procesos de calidad apropiados y asegurar que los mismos sean respetados.
- Debería ayudar a desarrollar una "cultura de calidad" donde la calidad es vista como una responsabilidad de todos y cada uno.
- Implica insertar en cada una de las actividades acciones tendientes a detectar lo más tempranamente posible oportunidades de mejora sobre el producto y sobre el proceso.
- Para materializarlo en forma concreta hay que tener en cuenta las siguientes consideraciones (<u>Reporte del grupo de aseguramiento de calidad (GAC)</u>):
    - No debería reportar la gente que hace calidad al mismo gerente que reporta los proyectos (porque le quita independencia y libertad).
    - El grupo de aseguramiento de calidad debería depender del gerente general.
    - Cuando sea posible, el grupo de aseguramiento de calidad debería reportar a alguien realmente interesado en el aseguramiento de calidad.
- <u>Actividades de administración de Calidad del Software</u>:
    - Aseguramiento de calidad: establecer estándares y procedimientos de calidad. elegir contra que estándares se van a efectuar las comparaciones.
    - Planificación de calidad: se seleccionan los procedimientos y estándares de calidad para un proyecto y particular, y se los modifica si fuera necesario.

La administración de calidad debería estar separada de la administración de proyectos para asegurar

- Control de calidad: se ejecuta el control de calidad de acuerdo a los procedimientos y estándares de calidad elegidos. se ejecutan las actividades definidas en la planificación para ver en qué situación está el proyecto.

Procesos basados en calidad



- El trabajo de las personas encargadas de la calidad en el desarrollo de software implica definir y establecer los procesos que se seguirán para garantizar la calidad del producto final.
- Una vez que se han establecido los procesos, es importante evaluar continuamente su efectividad y calidad.
- En función de los resultados de la evaluación del producto, se toman decisiones sobre la mejora del proceso. Si se identifican deficiencias o problemas en el producto, se realiza una revisión exhaustiva del proceso correspondiente y se implementan mejoras para corregir los errores y evitar que se repitan en el futuro.
- Por otro lado, si el producto de software cumple con los estándares de calidad y se considera exitoso, el proceso utilizado se estandariza. Esto implica documentar y comunicar de manera efectiva los pasos, las actividades y las buenas prácticas empleadas durante el desarrollo del producto.

Calidad de Procesos en la Práctica

- Definir procesos estándares.
- Monitorear el proceso de desarrollo para asegurar que los estándares sean respetados.
- Reportar en el proceso a la Administración de Proyectos y al responsable del software.
- No use prácticas inapropiadas simplemente porque se han establecido los estándares.

Estándares y Aseguramiento de Calidad

- Los estándares son la clave para la administración de calidad efectiva.
- Pueden ser estándares internacionales, nacionales, organizacionales o de proyecto.
- Estándares de Producto definen las características que todos componentes deberían exhibir, ej. estilos de programación común.
- Estándares de Procesos definen cómo deberían ser implementados los procesos de software.

Planificación de la Calidad

- Un plan de calidad define los productos de calidad deseados y como serán evaluados , y define los atributos de calidad más significativos.
- El plan de calidad debería definir el proceso de evaluación de la calidad.
- Define cuales estándares organizacionales deberían ser aplicados , como así también si es necesario utilizar nuevos estándares.

Control de Calidad

- Este implica el control del proceso de desarrollo para asegurar que se siguen los estándares y procedimientos .
- Existen dos enfoques para el control de calidad:
    - Revisiones de Calidad; Evaluaciones de Software Automáticas y mediciones.

Revisiones de Calidad

- Este es el principal método de validación de la calidad de un proceso o un producto.
- Un grupo examina parte de un proceso o producto y su documentación para encontrar potenciales problemas.
- Existen diferentes tipos de revisiones con diferentes objetivos
- Inspecciones para remoción de defectos (producto); ▫ Revisiones para evaluación de progreso (producto y proceso); ▫ Revisiones de Calidad (producto y estándares).

MEJORA DE PROCESOS

- Los modelos de mejora son recomendaciones o estándares para encarar un proyecto de mejora de un proceso.
- El propósito de un modelo de mejora es tomar el proceso de una organización y elaborar un proyecto, cuyo resultado va a ser un proceso mejorado.

IDEAL (INITATING, DIAGNOSING, ESTABLISHING, ACTING, LEARNING)

- INITIATING (Inicio): Se busca un sponsoreo en la organización. Esto es importante, puesto que los proyectos de mejoras de proceso no suelen ser tomados como prioridad a menudo en las organizaciones.
- DIAGNOSING (Diagnostico): Se realiza un diagnóstico para determinar cuál es el punto de partida del proyecto. Se realiza un Análisis de brecha: se analiza en donde esta parada la organización y a qué objetivo quiere apuntar.
    - Para el análisis de brecha entran en juego los Modelos de calidad, que se usan como referencia para bajar lineamientos que el proceso debe seguir para llegar a un cierto objetivo.
- ESTABLISHING (Establecimiento): Se planifica y se establece un plan de mejora detallado. Se definen los objetivos, se identifican las actividades necesarias y se asignan los recursos necesarios.
- ACTING (Actuar): Se ejecuta el plan de acción y las estrategias definidas. Probamos el proceso definido en algún proyecto (pruebas pilotos).
- LEARNING (Aprender): Si el resultado es positivo, lo extrapolamos a toda la organización. Se documentan los resultados del plan de acción y se vuelve a ejecutar el modelo con las lecciones aprendidas y para identificar oportunidades de mejora.

CMMI (Capability Maturity Model Integration // modelo de madurez y capacidad integrada**)**

- Surgió específicamente para empresas, organizaciones que hacen específicamente software. Modelo que se usa de referencia en base a los objetivos que quieren alcanzarse.
- El CMMI dice el qué no el cómo, es un modelo descriptivo no prescriptivo.

- El objetivo principal es proporcionar una guía para mejorar la capacidad y madurez de los procesos organizacionales. El modelo se basa en buenas prácticas recopiladas de la industria.
- Se estructura en niveles de madurez y áreas de proceso.
- Es uno de los modelos mas implementados en todo el mundo.
- No es una norma y no se “certifica”, solo se evalúa a través de profesionales reconocidos por el SEI como Lead Appraisers.
- Tiene 3 dominios o ámbitos de mejora (Constelaciones):
    - CMMI DEV: Provee la guía para medir, monitorizar y administrar los procesos de desarrollo. Se enfoca específicamente en el desarrollo de software
    - CMMI SVC: “ “ para entregar servicios, internos o externos.
    - CMMI ACQ: “ “ para administrar, seleccionar y adquirir productos o servicios.

Representaciones CMMI (Aparecen dos formas de mejorar, evolucionar con el proceso.)

- Por etapas: En esta representación, se organizan los niveles de madurez de la organización en etapas predefinidas. Cada nivel de madurez representa un conjunto de prácticas y capacidades específicas que deben alcanzarse para avanzar al siguiente nivel.
    - CMM originalmente dividía las org en “Maduras” (lvl1) e “Inmaduras” (lvl 2-5), mientras mas maduras, mas capacidad de cumplir con sus objetivos.
    - Nivel 1: Inicial: La organización tiene procesos ad hoc y no estándarizados.
    - Nivel 2: Gestionado: La organización establece prácticas de gestión básicas y utiliza enfoques más disciplinados para el desarrollo de proyectos.
    - Nivel 3: Definido: La organización establece y documenta procesos estandarizados y repetibles.
    - Nivel 4: Cuantitativamente gestionado: La organización establece mediciones cuantitativas para el control y mejora de los procesos.
    - Nivel 5: Optimizado: La organización enfoca en la mejora continua y la optimización de los procesos.
- Continua (6 niveles, del 0-5): Esta representación lo que hace es elegir áreas de proceso dentro de las que el modelo te ofrece (son 22 en la última versión) y yo elijo cuales son los procesos quiero mejorar por separado, entonces en lugar de medir la madurez de toda la organización, se mide la capacidad de un proceso en particular.
    - Mide áreas individuales.
    - En vez de medir las MADUREZ de la organización, mide la CAPACIDAD de un proceso en particular (puede ser 0 en Gest de Conf de soft y 2 en gest de requer).
    - La evaluación de la capacidad se realiza en una escala de 0 a 5, donde cada nivel de capacidad indica el grado de implementación y desempeño de las prácticas específicas de esa área.

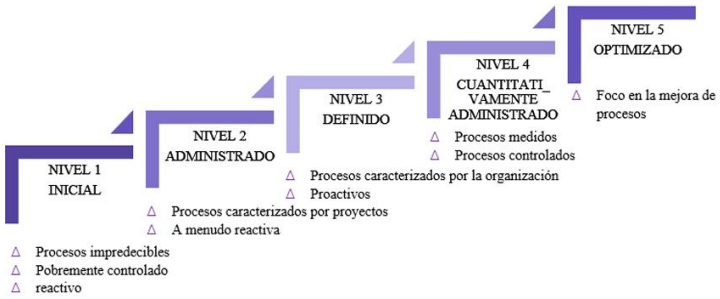
CMMI ¿ROLES Y GRUPOS?

- Cuando CMMI se refiere a grupos se refiere a la existencia de roles que cubren ciertas tareas. (Grupos no es necesariamente que haya más de una persona, es alguien que lo haga)

- Los roles se adaptan al tamaño de la organización y a las expectativas de madurez que la organización quiere llegar.

CMMI: Niveles de la Representación por Etapas (Los lvl son acumulativos, si sos 3, sos 2 y 1)

Mientras mas madures tenemos, mas productivos y menos riesgos tenemos.



- NIVEL 1: INICIAL No existe visibilidad respecto del proceso. No se sabe cuándo termina, cuánto cuesta ni la calidad de lo que se obtiene. Son organizaciones caracterizadas por “apagar incendios”. Estas organizaciones tienen actitud reactiva: buscan como atacar el problema una vez sucedió, en vez de buscar prevenirlo. Son organizaciones inmaduras. 2.
- NIVEL 2: ADMINISTRADO En este nivel, la organización comienza a tener controles básicos para la gestión de proyectos y procesos. Se busca una mayor estandarización y se inicia la recopilación de métricas para el seguimiento y la toma de decisiones. 3.
- NIVEL 3: DEFINIDO En este nivel, la organización ha establecido y documentado procesos definidos y estandarizados. Se definen roles y responsabilidades, y existe retroalimentación.
- NIVEL 4: CUANTITATIVAMENTE ADMINISTRADO En este nivel, la organización tiene la capacidad de cuantificar y medir su rendimiento. Se establecen métricas, y se recopilan datos para analizar el desempeño de los procesos y procedimientos.
- NIVEL 5: OPTIMIZADO En este nivel, la organización se enfoca en la mejora continua y en la optimización de sus procesos.

Si alcanzo un nivel 2 de CMMI puedo decir que mi organización tiene madurez para administrar sus proyectos y que el resultado de sus proyectos va a ser un producto de software de lo que se sabe que se espera de él.

Areas de proceso del nivel 2:

Se tienen que cumplir todos los objetivos que tiene cada área de proceso del nivel. Si un área de proceso se cae (no cumple por lo menos un objetivo) y si se cae el área se cae el nivel.

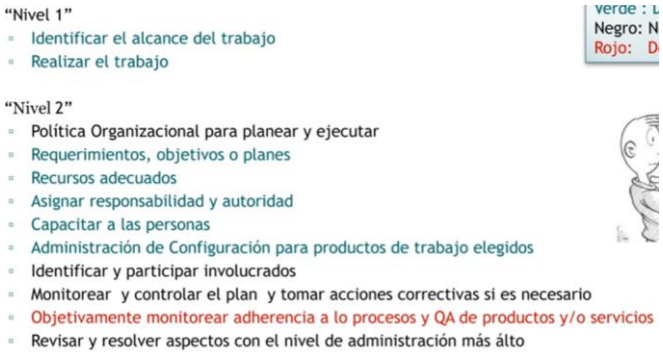
- Administración de Requerimientos.
- Planeamiento de Proyectos.
- Monitoreo y Control de Proyectos.
- Administración de Acuerdo con el Proveedor.
- Medición y Análisis
- Aseguramiento de Calidad del Proceso y del Producto.
- Administración de Configuración.

Puntos clave

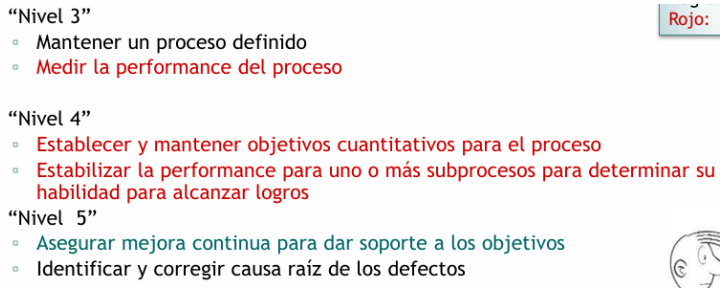
- El software puede analizarse desde varias perspectivas: como proceso, como producto, la calidad también.
- La calidad del software es difícil de medir.
- El software como proceso es el fundamento para mejorar la calidad.
- Trabajar con calidad es más barato.

- La mejora de procesos exitosa requiere compromiso organizacional y cambio organizacional.
- Existen varios modelos disponibles para dar soporte a los esfuerzos de mejora.
- La mejora de procesos en el software ha demostrado retornos de inversión sustanciales.

CMMI cara a cara con Ágil



En el nivel 2 chocan con el monitoreo ya que se refiere a auditorias independientes, donde ágil no "esta de acuerdo" ya que plantea que todo se resuelve dentro del equipo.
Mientras subimos de nivel empiezan a haber problemas con las métricas y las mejoras de procesos entre equipos y los métodos.
CMMI dice que vos tenes que definir un proceso y que lo tenes que cumplir, entonces cuando tenes que hacer una evaluación o una auditoria, la auditoria no solo mide que tengas los productos que dijiste que ibas a tener si no también que los hayas construido con el proceso que lo ibas a hacer. Y eso ágil no lo hace porque busca SW funcionando y el cliente contento, no se evalúa si respetamos el proceso (no significa que no se haga).



AUDITORIAS

- Es una actividad incluida dentro de las disciplinas de soporte que implica una evaluación independiente y objetiva que busca
  - Realizar controles apropiados del software y el proceso de desarrollo.
  - Asegurar el cumplimiento de los estándares y procedimientos para el software y el proceso.
  - Asegurar que los defectos en el producto, proceso o estándares son informados a la gerencia para que puedan ser solucionados.
- Definición IEEE:
  - Evaluación independiente de los productos o procesos de software para asegurar el cumplimiento con estándares, lineamientos, especificaciones y

procedimientos, basada en un criterio objetivo incluyendo documentación que especifique:
    - La forma o contenido de los productos a ser desarrollados
    - El proceso por el cual los productos son desarrollados
    - Cómo debería medirse el cumplimiento con estándares o lineamientos.
- Beneficios: → Resultado: Mejores productos conllevan a clientes satisfechos y crecimiento del negocio
    - Evaluar el cumplimiento del proceso de desarrollo
    - Determinar la implementación efectiva de:
        - El proceso de desarrollo organizacional
        - El proceso de desarrollo del proyecto
        - Las actividades de soporte
    - Dar visibilidad a la gerencia sobre los procesos de trabajo

Tipos de auditorías de software

Auditorías de Proyecto

- Valida el cumplimiento del proceso de desarrollo. Es responsable de evaluar si el proyecto se ejecutó en base al proceso que se dijo que se iba a ejecutar.
- Las auditorías de proyecto se llevan a cabo de acuerdo a lo establecido en el PACS.
- El PACS debería indicar la persona responsable de realizar estas auditorías.
- Las inspecciones de software y las revisiones de la documentación de diseño y prueba deberían incluirse en esta auditoría.
- El objetivo de esta auditoría es verificar objetivamente la consistencia del producto a medida que evoluciona a lo largo del proceso de desarrollo, determinando que:
    - Las interfaces de HW y SW sean consistentes con los requerimientos de diseño en la ERS.
    - Los requerimientos funcionales de la ERS se validan en el Plan De Verificación y Validación de Software.
    - El diseño del producto, a medida que DDS evoluciona, satisface los requerimientos funcionales de la ERS.
    - El código es consistente con el DDS.

Auditoría de Configuración Funcional (Valida)

- Compara el software que se ha construido con los requerimientos de software especificados en la ERS.
- Su propósito es asegurar que el código implementa sólo y completamente los requerimientos y las capacidades funcionales descriptos en la ERS.
- El responsable de QA deberá validar si la matriz de rastreabilidad está actualizada.

Auditoría de configuración física (Verifica)

- Compara el código con la documentación de soporte.
- Su propósito es asegurar que la documentación que se entregará es consistente y describe correctamente al código desarrollado.
- El PACS debería indicar la persona responsable de realizar la auditoría física.
- El software podrá entregarse sólo cuando se hayan arreglado las desviaciones encontradas.

Roles

- Auditado

    - acuerda la fecha de la auditoría
    - participa de la auditoría
    - proporciona evidencia al auditor.
    - contesta al reporte de auditoría
    - propone el plan de acción para deficiencias citadas en el reporte
    - comunica el cumplimiento del plan de acción.
- Auditor (No es del proyecto)
    - acuerda la fecha de la auditoría
    - comunica el alcance de la auditoría
    - recolecta y analiza la evidencia objetiva que es relevante y suficiente para tomar conclusiones acerca del proyecto auditado
    - realiza la auditoría
    - prepara el reporte
    - realiza el seguimiento de los planes de acción acordados con el auditado.
- Gerente de SQA
    - prepara el plan de auditorías
    - calcula el costo de las auditorías
    - asigna los recursos.
    - responsable de resolver las no-conformidades

Proceso de auditoría (etapas)



- Preparación y planificación: Se planifica y prepara la auditoria en forma conjunta entre el auditado y el auditor. Generalmente es el líder de proyecto quien la convoca. Estas no son sorpresas.
- Ejecución: Durante la ejecución, el auditor pide documentación y hace preguntas. Busca evidencia objetiva (lo que está documentado), y subjetiva (lo que el equipo expresa que hace).
- Análisis y reporte de resultado: Se analiza la documentación, se prepara un reporte y se lo entrega al auditado.
- Seguimiento: Dependiendo de cómo funcione el acuerdo entre el auditado y el auditor, el auditor puede hacer un seguimiento de las desviaciones que encontró hasta que considere que han sido resueltas.

Técnicas y herramientas

Checklist, Muestreo, Revisión de registros, Herramientas automatizadas.

El contenido general del checklist sería:
- Fecha de la auditoría
- Lista de auditados (identificando el rol)
- Nombre del auditor
- Nombre del proyecto
- Fase actual del proyecto (si aplica)
- Objetivo y alcance de la auditoría
- Lista de preguntas

Análisis y Reporte de Resultados

- Reporte de auditoría (contenido)
    - Identificación de la auditoría, Fecha de la auditoría, Auditor, Auditados, Nombre del proyecto auditado, Fase actual del proyecto, Lista de resultados, Comentarios, Solicitud de planes de acción.
- Tipos de resultado de la auditoria:
    - Buenas prácticas: practica, procedimiento o instrucción que se ha desarrollado mucho mejor de lo esperado.
    - Desviaciones: requieren un plan de acción por parte del auditado. Cualquier cosa que no se hizo como el proceso indicaba que debía hacerse.
    - Observaciones: condiciones que deberían mejorarse pero que no requieren un plan de acción. Cosas que advierte el auditor que no llegan a ser desviaciones.

Métricas de auditoría (algunos ejemplos)

- Esfuerzo por auditoría: Mide la cantidad de tiempo y recursos dedicados a la realización de la auditoría. Puede ser expresado en horas o días de trabajo.
- Cantidad de desviaciones: Registra el total de desviaciones identificadas durante la auditoría.
- Duración de auditoría: Mide el tiempo total que lleva realizar la auditoría, desde la planificación hasta la presentación de los resultados.
- Cantidad de desviaciones clasificadas por área de proceso (PA) de CMMI: Desglosa las desviaciones identificadas según las diferentes áreas de proceso definidas por el modelo CMMI.

Revisiones Técnicas

VERIFICACIÓN Y VALIDACIÓN

- Es un proceso de ciclo de vida completo (SE HACE EN TODAS LAS ETAPAS)
- Inicia con las revisiones de los requerimientos y continúa con las revisiones del diseño, inspecciones del código hasta la prueba.
- Debe planificarse
    - Es un proceso caro
    - Debemos empezar en etapas tempranas

FALLA

- Es un error en un producto de trabajo (puede ser error o defecto)
- Producto de trabajo: salida de cualquier actividad correspondiente al ciclo de vida de desarrollo
- Existen por
    - Problemas de comunicación
    - Limitaciones de memoria (el software no administra correctamente la memoria disponible del sistema.)
    - Complejidad inherente al producto que se desarrolla.
- A medida que avanzo en el proyecto cuesta más reparar una falla (crece exponencial)
- Problemas del retrabajo
    - El retrabajo evitable corresponde al 40-50% del desarrollo
    - Un defecto me da mucho retrabajo (por eso malo)

Para detectar falla en requerimientos no hago testing hago verificaron estática (revisión)

PRINCIPIOS DE VALIDAR Y VERIFICAR

- La prevención es mejor que la cura.
- Evitar es más efectivo que eliminar.
- La retroalimentación enseña efectivamente.
- Priorizar lo rentable.
- Olvidarse de la perfección, no se puede conseguir.
- Enseñar a pescar, en lugar de dar el pescado.

APROXIMACIONES COMPLEMENTARIAS

- Pruebas de software.
- Revisiones técnicas.
    - Es una actividad de verificación y validación que se aplica en varios momentos del desarrollo.

- Busca detectar defectos y corregirlos en etapas tempranas de desarrollo
- No requieren que el programa se ejecute (actividad estática).
- Puede inspeccionar cualquier artefacto de software (código, req, diseño, planes, arquitectura, estimaciones, etc).
- Se necesita revisar el trabajo técnico porque errar es humano (lo hace un colega, ya que la persona que origina el error muchas veces le es complicado darse cuenta del mismo).
- Se hace sin poner en juicio al autor del artefacto, sólo el artefacto en sí
- Motiva a realizar un mejor trabajo y evita retrabajo.
- Muchas veces esto es lo que usa agile para evitar auditorias "es una forma de auditar".
- Asegura la calidad de los entregables del producto y del proyecto.

VENTAJAS	DESVENTAJAS
Pueden descubrirse muchos errores	Es difícil introducir las inspecciones formales
Pueden inspeccionarse versiones incompletas	Sobrecargan al inicio los costos y conducen a un ahorro sólo después de que los equipos adquieran experiencia en su uso.
Pueden considerarse otro atributos de calidad	Requieren tiempo para organizarse y parecen ralentizar el proceso de Desarrollo

- Tienen asociados una serie de costos.

Infraestructura	Operacionales	Adicionales
- Entrenamiento - Desarrollo/ajuste de plantillas, informes y guías de lectura - Implantación de programas de medición - Herramientas de soporte	- Tiempo individual y grupal - Tiempo en completar informes.	- Preparar material, arreglar calendario, recolectar datos, etc. - Tiempo dedicado a la mejora de calidad

EXISTEN DOS TIPOS DE REVISIONES TÉCNICAS

- Walkthrough o recorrido
  - Técnica de análisis estático en la que un diseñador o programador dirige miembros del equipo de desarrollo y otras partes interesadas mediante un producto de software
  - Los participantes formulan preguntas y realizan comentarios acerca de posibles errores, violación de estándares de desarrollo y otros problemas
  - Se realiza mediante reuniones informales entre colegas, sin un proceso rígido ni controles formales.
- Objetivos
  - Mínima Sobrecarga
  - Capacitación de Desarrolladores
  - Rápido retorno
- Características
  - Poca o ninguna preparación
  - Proceso informal
  - No hay mediciones o métricas para aprender y dejar registros
  - Es una de las técnicas más elegidas en los enfoques agiles
  - No hay FTR (revisión técnica formal)
- Inspecciones
  - Tiene un proceso formal y roles definidos para cada participante de la revisión

- Se utiliza un checklist, que guía la revisión para asegurarnos que no se olviden cosas importantes a controlar
- Se toman métricas y al finalizar se realiza un reporte de la revisión al final de la inspección para analizar los defectos encontrados o Son procesos time-boxed, con tiempos previamente definidos, y requieren un alto esfuerzo intelectual por parte de los participantes.
- Es una actividad que garantiza la calidad del software, cuyo éxito depende de la planificación.

- Objetivos
  - Detectar y remover todos los defectos de forma eficiente y efectiva.
  - Verificar que el software alcanza sus requisitos.
  - Garantizar que el software ha sido representado según ciertos estándares.
  - Conseguir un software desarollado de manera uniforme.
  - Hacer que los proyectos sean más manejables.

ROLES QUE PARTICIPAN EN LAS INSPECCIONES

Rol	Responsabilidad
Autor	•Creador o encargado de mantener el producto que va a ser inspeccionado. •Inicia el proceso asignando un moderador y designa junto al moderador el resto de los roles •Entrega el producto a ser inspeccionado al moderador. •Reporta el tiempo de retrabajo y el nro. total de defectos al moderador.
Moderador	•Planifica y lidera la revisión. •Trabaja junto al autor para seleccionar el resto de los roles. •Entrega el producto a inspeccionar a los inspectores con tiempo (48hs) antes de la reunión. •Coordina la reunión asegurándose que no hay conductas inapropiadas • Hacer seguimiento de los defectos reportados.
Lector	Lee el producto a ser inspeccionado.
Anotador	Registra los hallazgos de la revisión
Inspector	Examina el producto antes de la reunión para encontrar defectos. Registra sus tiempos de preparación.

CUESTIONES A TENER EN CUENTA SOBRE REVISIONES TÉCNICA

SON	NO SON
La forma más barata y efectiva de encontrar fallas	Utilizadas para encontrar soluciones a las fallas
Una buena forma de proveer Metricas al proyecto Conocimiento cruzado Trabajo en grupo	Usadas para obtener la aprobación de un producto de trabajo
Un método probado para mejorar la calidad del producto	Usadas para evaluar el desempeño de las personas

ETAPAS DEL PROCESO DE INSPECCIÓN (Todo esto sumado no puede durar más de 2 horas)

- Planificación
  - El moderador, a pedido del autor, planifica la inspección definiendo el lugar, el tiempo de duración y los roles.
- Visión general
  - Etapa opcional en la que el autor presenta el proceso y producto a inspeccionar
- Preparación
  - Cada rol adquiere una copia del producto de trabajo que deberá leer y analizar, con el fin de encontrar potenciales defectos de forma individual
  - Permite que la reunión de inspección sea más productiva.
- Reunión de inspección
  - El equipo realiza un análisis para recolectar los potenciales defectos previos y descartar falsos positivos.
  - El lector lee el producto de trabajo y los inspectores comparten los defectos encontrados, los cuales son registrados por el anotador.

    - La reunión finaliza con una conclusión acerca de si se acepta, se rechaza o sea acepta, pero de forma provisoria el producto de trabajo inspeccionado y se realiza un informe (que se revisó, quién, que se descubrió, conclusión)
- Corrección
    - Finalizada la reunión, el autor realiza las correcciones de los defectos encontradas.
- Seguimiento
    - En caso de que los defectos sean graves, se realiza nuevamente una inspección
    - De lo contrario, si el defecto era muy simple, el autor simplemente lo corrige.
    - Otros defectos que no implican una re-inspección, pueden implicar que el autor se reúna con el moderador únicamente para tratar las correcciones realizadas.

DOCUMENTOS Y REVISORES SUGERIDOS

Tipo de documento	Revisores sugeridos
Arquitectura o Diseño de alto nivel	Arquitecto, analista de requerimientos, diseñador, lider de proyecto, testers.
Diseño detallado	Diseñador, arquitecto, programadores, testers
Planes de proyecto	Líder de proyecto, stakeholders, representante de ventas o marketing, líder técnico, representante del área de calidad,
Especificacion de requerimientos	Analista de requerimientos, líder de proyecto, arquitecto, diseñador, testers, representante de ventas y/o marketing
Codigo fuente	Programador, diseñador, testers, analista de requerimientos
Plan de testing	Tester, programador, arquitecto, diseñador, representante del área de calidad, analista de requerimientos

TESTING (En procesos definidos - Tradicional)

- Tarea que se realiza en el ámbito del aseguramiento de calidad de software, en conjunto con el control de calidad del proceso y del producto
- Luego de desarrollar software, las actividades de testing revelan la calidad de software en sí, es decir, si el producto satisface con los requerimientos definidos por el cliente.

¿QUÉ ES EL TESTING/PRUEBA?

- Es un proceso destructivo que trata de encontrar defectos en el software, su objetivo no es asegurar que el software funcione, sino encontrar defectos, para lo que asumimos que en el software HAY defectos.
- No debemos pensar que no hay defectos ni que debemos verificar que no los hay, sino que empezamos con la hipótesis o la actitud negativa de que el software es incorrecto.
- Decimos entonces, que el test va a ser exitoso cuando encontremos defectos.
- Un desarrollo exitoso nos conduce a un test no exitoso.
- Mundialmente: del 30% a 50% del costo del software (lo hace confiable).

ERROR	vs	DEFECTO
El error se descubre a partir de técnicas específicas que me permiten encontrar los mismos dentro de la misma etapa en la que estoy trabajando		Los defectos nos demuestran un error no detectado que se trasladó a una etapa siguiente

- En el testing encontramos defectos, ya que justamente se encuentran cosas incorrectas que se realizaron en la etapa de implementación que es la etapa anterior.

- Por esto siempre es mejor encontrar errores antes que defectos.
- Cuando hablamos de defectos encontrados durante el testing, debemos considerar dos aspectos que nos van a definir el accionar que vamos a realizar sobre estos:

SEVERIDAD	PRIORIDAD
Tiene que ver con la gravedad del defecto que encontré. Un defecto puede ser **bloqueante** y no permitirme seguir con el caso de prueba, **critico** se refiere a un defecto que compromete la ejecución del caso de prueba y asi sucesivamente va disminuyendo desde **mayor**, **menor**, hasta **cosmético** (que podría ser alguna cuestión de sintaxis, ortografía, visualización, etc.)	Urgencia que tenemos para resolver este defecto. Se podría intuir que, según la severidad de los defectos, va a haber tal o cual prioridad, pero esto no siempre es asi. Esto va a depender del contexto en el cual nos encontramos, es por esto la importancia de tener los dos aspectos identificados.

Niveles de prueba

- PRUEBAS UNITARIAS (en el momento de la implementación):
  - Las hace el desarrollador y están incluidas en el DoD.
  - Son pruebas que se realizan sobre un componente, de manera
  - independiente a los otros.
  - Se hacen teniendo acceso al código fuente, y se pueden usar herramientas para realizarlas (automatización, depuración).
  - Se suelen reparar los errores apenas se encuentran, sin registrarlos formalmente.
- PRUEBA DE INTEGRACIÓN (depende el equipo como se llevan adelante y quienes son los responsables):
  - El propósito de la ejecución de estas pruebas es verificar el funcionamiento de dos o más componentes juntos que se relacionen entre ellos, es decir, clases en las cuales existe una interacción a modo de peticiones, a través de sus interfaces (boundaries).
  - El resultado de estas pruebas es el build (el CI o continuous integration), el cual luego se envía al equipo de testing y es lo que se prueba en siguientes etapas.
  - Se suele llevar a cabo una prueba de integración incremental, desde lo más general (que abarca más componentes) a lo más específico (top-down) o desde lo más específico a lo más general (bottom-up).
- PRUEBAS DE SISTEMA(en el workflow de prueba):
  - Son más amplias, no solo pruebo la integración entre dos componentes, sino el sistema en toda su escala.
  - Busca asegurarse que el sistema en su totalidad funcione de manera satisfactoria. Empiezan a impactar otras cuestiones que tienen que ver no solo con requerimientos funcionales, sino también con requerimientos no funcionales.
  - El ambiente de prueba tiene que ser lo más parecido al entorno de producción.
  - Se recomiendan que sean realizadas por personas ajenas al desarrollo de los programas (obligatoriamente).
  - Estas pruebas se llevan adelante siguiendo un proceso sistemático y metodológico, que permite encontrar defectos que puedan ser reproducibles y reportar de qué manera ocurre el defecto detectado, es decir, cual es el camino de pasos que hay que seguir para encontrar el error.
- PRUEBAS ACEPTACIÓN DE USUARIO(en el workflow de despliegue):

    - se realizan en el despliegue y debería realizarlas el usuario para verificar que se cumple con lo que el mismo requirió.
    - Busca que el cliente/usuario se familiarice con el producto, generando comodidad y confianza sobre el mismo (su objetivo principal no es encontrar fallas).
    - También busca reproducir un entorno de producción. Comprende tanto la prueba realizada por el usuario en ambiente de laboratorio (pruebas alfa), como la prueba en ambientes de trabajo reales (pruebas beta). En la teoría, los usuarios arman sus pruebas de usuario. En la práctica, las arma Testing.
    - En la teoría, además del usuario, deben estar presentes el gerente de testing, el líder del proyecto y empleados con roles funcionales.
    - En Agile, además del usuario, deben estar todos los miembros del equipo (Sprint Review).

AMBIENTES PARA CONSTRUCCIÓN DEL SOFTWARE

Son todos los recursos, tanto hardware como software, que se requieren para poder trabajar con el producto. Los ambientes para la construcción del software se refieren a los diferentes entornos utilizados en el ciclo de vida del desarrollo de software. Cada ambiente cumple un propósito específico y se utiliza en diferentes etapas del proceso de desarrollo y despliegue.

- Ambiente de desarrollo:
    - Es donde los desarrolladores crean, prueban y depuran el software. Es un entorno local o en red utilizado por los desarrolladores para escribir y probar el código antes de integrarlo con el resto del sistema.
    - Los programadores pueden utilizar herramientas de desarrollo, depuración y pruebas para garantizar que el software cumpla con los requisitos establecidos.
    - Este ambiente suele ser flexible y permite a los desarrolladores experimentar y probar nuevas ideas sin afectar los entornos de producción. Las pruebas unitarias se llevan a cabo en el ambiente de desarrollo.
- Ambiente de Prueba:
    - Es el ambiente que utilizan los testers para llevar a cabo las pruebas, y los desarrolladores no deben tener acceso.
    - Es donde se llevan a cabo pruebas exhaustivas del software desarrollado. Aquí se realizan pruebas de integración, pruebas funcionales, pruebas de rendimiento y otras pruebas relevantes para verificar que el software cumpla con los requisitos establecidos y funcione correctamente.
    - Suele ser una réplica o tener características parecidas al entorno de producción, pero sin afectar a los usuarios finales y sin TODAS las características (ya que es caro tener ambientes iguales al de producción).
- Ambiente de Pre-Producción:
    - Debería tener las mismas características que el ambiente productivo para poder comprobar que el producto funcionará una vez desplegado en producción de manera correcta.
    - En este ambiente se realizan las pruebas de aceptación.
    - El objetivo es validar y verificar que el software esté listo para ser implementado en el entorno de producción sin problemas significativos.
    - Las pruebas de sistema se llevan a cabo en el ambiente de preproducción
- Ambiente de Producción:

- Es el ambiente en el que el software está funcionando y es accesible a los usuarios finales.
- Es el entorno real en el que el software se utiliza para realizar las tareas y funciones para las que fue diseñado.
- Este ambiente suele ser altamente controlado y está configurado para ser escalable, seguro y confiable.
- Se implementan medidas de respaldo y recuperación ante desastres para garantizar la disponibilidad continua del software.

Artefactos de Testing

PLAN DE PRUEBA

- Es análogo al plan de proyecto que se elabora en el enfoque tradicional (no forma parte de ese plan). En este plan, se definen: Datos necesarios para las pruebas - Recursos destinados a las pruebas. - Herramientas a utilizar. - Si se aplicará regresión (o no).
- Para la confección de este plan no es necesario el código, sino que sólo se necesitan los requerimientos, en el formato que sea que se encuentren (ERS o Casos de Uso en métodos tradicionales, User Stories en Agile, etc.)

CASO DE PRUEBA

- Es el artefacto más importante del testing.
- Tiene que ver con un conjunto de condiciones o variables que nos van a permitir determinar si el software está funcionando correctamente o no.

OBJETIVO	**descubrir errores**
CRITERIO	**en forma completa**
RESTRICCIÓN	**con el mínimo de esfuerzo y tiempo**

- La buena definición de casos de prueba nos ayuda a reproducir defectos.
  - Se deben explicitar las condiciones de inicio (las cuales fijan una situación particular), y los datos utilizados en ese escenario.
- Se confeccionan con el objetivo de descubrir la mayor cantidad de defectos y minimizar el esfuerzo y tiempo para elaborarlos y ejecutarlos.
- Mientras no cambien los requerimientos, estos casos pueden ser utilizados las veces que sea necesario.
- La característica más importante de los casos de prueba es ser REPRODUCIBLES. Si se encuentra un defecto y no es reproducible a través de un caso de prueba, no es un defecto.

El testing exhaustivo de TODAS las opciones del software no es posible. Para esto vamos a usar ESTRATEGIAS DE PRUEBA, vamos a conducir el esfuerzo hacia lo importante.

Se deben definir casos de prueba que apunten a valores limites o esquinas, es decir, a los lugares que son más propensos de tener errores/defectos. Probamos en los "limites".

<u>Derivación de casos de prueba</u>

- Desde documentos del cliente
- Desde información de relevamiento.
- Desde requerimientos. →

- Deberían hacerse los casos de prueba a partir de los requerimientos.
- Sin requerimientos bien detallados, todo es más difícil.
- Muchas veces se pueden utilizar los casos de prueba como especificación de requerimientos. Por eso es que deberían estar bien detallados.
- Si el caso de prueba no esta actualizado no se puede hacer la prueba.

- Desde especificaciones de programación.
- Desde el código.

Ninguna técnica es completa, las distintas técnicas atacan distintos problemas y por ende es esencial combinarlas para complementarlas ventajas de cada una y tener una especificación más detallada. Los casos de prueba dependen del código a testear y se debe tener en cuenta la conjetura de defectos.

Condiciones de prueba

- Tienen que ver con el contexto del sistema que tengo que tener en cuenta para poder hacer los casos de prueba.
- Son la reacción esperada de un sistema frente a un estímulo particular, y estando el mismo constituido por las distintas entradas.
- Es algo que se enuncia ANTES de definir los casos de prueba, defino las condiciones y en función de eso defino los casos.
- Una condición de prueba debe ser probada por al menos un caso de prueba.

REPORTE DE INCIDENTES O DEFECTO

Luego de la ejecución de los casos de prueba, se elabora un reporte, indicando cuáles fueron los casos de prueba que han pasado, y aquellos en los que se ha encontrado defectos, indicando cuáles fueron esos defectos encontrados y cómo reproducirlos para su detección y corrección. Este artefacto permite a los desarrolladores corregir esos defectos, para enviarlos nuevamente a testing.

INFORME FINAL

Suele contener métricas e información final del incremento del producto, se reporta cuántos ciclos y la cantidad de errores por ciclo, métodos, tipos de prueba utilizados, etc. Este es el único artefacto que es negociable en algunas organizaciones informales, se acuerda en la contratación del trabajo. Tiene utilidades estadísticas.

ESTRATEGIAS/MÉTODOS DE PRUEBA

- El objetivo es abarcar la mayor cantidad de pruebas con el menor esfuerzo.
- El tiempo y el presupuesto es limitado. Hay que pasar por la mayor cantidad de funcionalidades con la menor cantidad de pruebas
- Cobertura: EL porcentaje de pruebas que se hacen, sobre el total de pruebas que deberían hacerse para cubrir todo el testing del producto.

Caja negra

- No se dispone de la estructura interna de la implementación, sino que se analizan las funcionalidades como una caja negra, en términos de entradas y salidas de esa “caja”.
- El proceso consiste en ingresar determinados datos a esa funcionalidad vista como caja negra, y luego comparar los resultados obtenidos con los resultados esperados.
- En transacciones no sólo se debe considerar todos los datos posibles, sino todas las secuencias posibles de transacciones previas.
- Se clasifican en:
  - Métodos basados en especificaciones: Son aquellos que se ejecutan utilizando la documentación de especificaciones realizadas del producto.

        - Partición de equivalencias: proceso sistemático que consiste en identificar clases de equivalencia, que definen subconjuntos de datos que producen un resultado equivalente.
        - Análisis de los valores limites: variante del anterior. Utilizar los limites o valores de borde de las clases de equivalencia para la definición de los casos de prueba.
    - Métodos basados en experiencia: la experiencia y los conocimientos del tester son fundamentales para determinar las entradas del sistema y analizar los resultados.
        - Adivinanza de defectos: enfoque basado en la intuición y experiencia para identificar pruebas que probablemente expongan defectos del software, elaborando una lista de defectos posibles o situaciones propensas a error y realizando pruebas a partir de esa lista.
        - Testing exploratorio: el tester mientras va probando el software, va aprendiendo a manejar el sistema y junto con su experiencia y creatividad, genera nuevas pruebas a ejecutar.

Caja blanca

- Se dispone del código (puede ser también pseudocódigo o un diagrama de flujo).
- Nos permiten diseñar casos de prueba, maximizando la cantidad de defectos con la menor cantidad de casos de prueba posibles.
- Tiene dos fallas:
    - La primera es que el número de caminos lógicos únicos puede ser sumamente grande, tomando muchísimo tiempo de probar absolutamente todas de ellas.
    - La segunda falla es que las pruebas de camino exhaustivas no aseguran que el programa sea el correcto para las especificaciones, tampoco detecta caminos faltantes ni determina errores de datos sensibles.
- Hay distintas coberturas con nivel diferente (la forma de recorrer los distintos caminos que provee el código para desarrollar una funcionalidad):
    - Cobertura de enunciados o caminos básicos:
        - Tiene como objetivo encontrar todos los caminos independientes de una funcionalidad que deben recorrerse (testear) al menos una vez, para probar cada uno de ellos con casos de prueba.
        - Permite obtener una métrica denominada complejidad ciclomática (M), que representa la cantidad de caminos independientes que posee una funcionalidad, y nos da un límite inferior para el número de casos de prueba que hay que construir, para ejecutar todas las instrucciones de la funcionalidad al menos una vez.
    - Cobertura de sentencias:
        - Sentencia: cualquier instrucción como asignación de variable, invocación de métodos, etc. Las estructuras de control NO son sentencias, son decisiones.
        - El objetivo es buscar la cantidad mínima de casos de pruebas que me permitan pasar, ejecutar o evaluar todas las sentencias.
    - Cobertura de decisión:
        - Estructura de control concreta, y dentro de cada decisión existen combinaciones que están unidas por operaciones booleanas de condiciones.
        - El objetivo es buscar la cantidad mínima de casos de prueba que me permitan evaluar todas las ramas de las decisiones.

- Cobertura de condición:
    - Son cada una de las evaluaciones lógicas dentro de una decisión, que están unidas por operadores lógicos.
    - Su objetivo es buscar la cantidad mínima de casos de pruebas que me permitan evaluar cada una de las condiciones, en su valor verdadero y en su valor falso, independientemente de que rama se tome en la decisión en la que se encuentre la condición.
- Cobertura de decisión condición
    - Su objetivo es evaluar las ramas verdaderas y falsas de las decisiones, y a su vez evaluar valores verdaderos y falsos de las condiciones.
- Cobertura múltiple
    - Su objetivo es evaluar el combinatorio de todas las condiciones, en todos sus valores de verdad posibles.

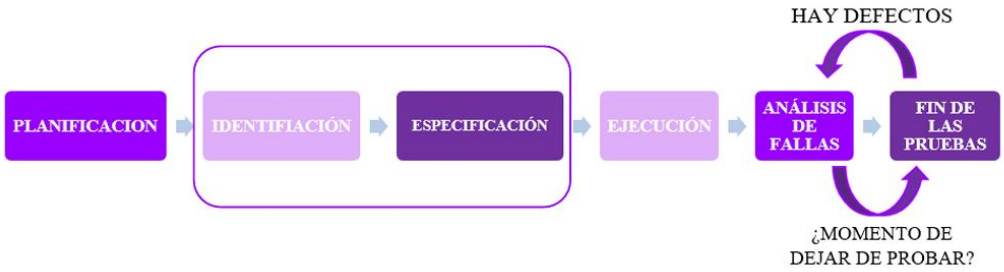
CICLO DE PRUEBA O CICLO DE TEST

- Un ciclo de pruebas abarca la ejecución de la totalidad de los casos de prueba establecidos aplicados a una misma versión del sistema a probar; ejecuto todos los casos de prueba, veo cuales fallaron, corrijo los defectos, y hago otro ciclo, lo que implica correr todos los casos de prueba en la nueva versión que ya tiene los defectos corregidos, esto en loop hasta que cumplamos con el criterio de aceptación que hayamos definido para el Testing.
- Empieza con la corrida del primer caso de prueba y termina cuando se ejecutaron todos los casos de prueba o cuando existe un error invalidante que permite que ese ciclo continue.

Regresión (todos los ciclos de prueba se van a ejecutar como si fueran el ciclo 0)

- Al concluir un ciclo de pruebas, y reemplazarse la versión del sistema sometido al mismo, debe realizarse una verificación total de la nueva versión, a fin de prevenir la introducción de nuevos defectos al intentar solucionar los detectados.
- La teoría nos dice que cuando solucionamos un defecto normalmente se introducen 2/3 defectos más. Es importante volver a verificar la nueva versión en su totalidad para asegurarnos de no haber introducido nuevos defectos. No siempre debemos hacer regresión, eso lo vamos a ir viendo a medida que testeamos.

PROCESO DE TESTING EN AMBIENTES TRADICIONALES

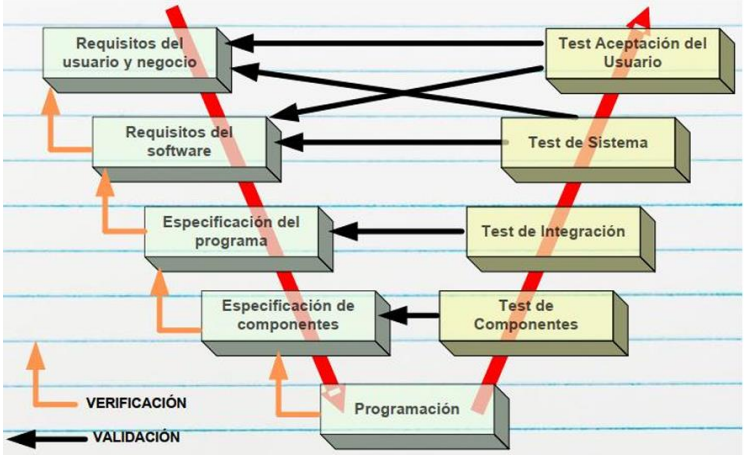


- PLANIFICACION: Se trata de la parte más estratégica del testing, se genera un Plan de Prueba (hablando de un proyecto tradicional).
- IDENTIFICACION Y ESPECIFICACION: Se definen y escriben los Casos de Prueba

- EJECUCION: Se lleva a cabo la ejecución de las pruebas (de manera manual o automatizada, o una combinación de ambas)
- ANALISIS DE FALLAS: Se analiza si la solución/resultados de la ejecución de los casos de prueba fueron o no los esperados. Si obtuvimos los resultados esperados, ocurre el FIN DE LAS PRUEBAS, si no cumplió con lo esperado voy a hacer un análisis de fallas para corregirlo.
- Este ciclo entre ejecución y análisis de fallas se ejecuta tantas veces como sea necesario para llegar al criterio de aceptación que va a determinar luego el FIN DE LAS PRUEBAS.

Modelo en V

- Se prueba de manera inversa a la que se desarrolla.
    - Se desarrolla de lo general a lo particular.
    - Se prueba de lo particular a lo general.



- Uno puede adelantar actividades del proceso de pruebas mientras el código se esta construyendo. De modo que cuando el código llegue a etapa de prueba ya prodia haber adelantada gran parte.
- Este modelo se utiliza para poder determinar cuándo se está en condiciones de realizar determinadas actividades, en función de la etapa de desarrollo en la que se encuentre el software.

PRINCIPIOS DEL TESTING (INVESTIGAR DESPUES DE DONDE SALEN https://www.youtube.com/watch?v=xePnnw7b71E)

- El testing muestra presencia de defectos (cuya presencia se asume) ayudando a que el software sea confiable (no asegura la calidad)
- Se debe testear de forma temprana (no debe empezar al terminar de codificar)
- El testing exhaustivo es imposible
- El testing depende del contexto
- Es una falacia decir que cuando termina el testing hay una ausencia de errores
- Agrupamiento de defectos
    - Los defectos en el software suelen agruparse en un conjunto limitado de áreas (el 80% de los defectos se agrupan en el 20% de la funcionalidad), por eso las pruebas deben priorizarse y enfocarse en ese conjunto
- Paradoja de pesticida

    - Cuando ejecutamos muchas veces los mismos casos de prueba, puede suceder que los mismos se ejecuten sin fallas, pero que los verdaderos defectos no se verifiquen y queden ocultos
- Un programador debería evitar probar su propio código (única excepción son pruebas unitarias)
    - Los propios desarrolladores “no quieren” encontrar sus propios defectos (se deja de lado el carácter destructivo)
    - Además, puede que el programa contenga errores por malentendidos del programador sobre el dominio, y si él mismo testea, no se van a detectar estos defectos.
- Una unidad de programación no debería probar sus propios desarrollo
- No pruebo que haga solo lo que debe hacer, busco también ver q pasa si hago lo que no debe hacer
- No planificar el esfuerzo de testing sobre la suposición de que no se van a encontrar defectos (debemos ir con la premisa que vamos a encontrar)

- La búsqueda de fallas puede ser visto como una crítica al producto y/o su autor (por el construir software requiere otra mentalidad a la de testear software)

¿Cuánto testing es suficiente?

- Es imposible probar todas las opciones posibles, el testing exhaustivo no es viable.
- Decidir cuánto testing es suficiente depende de una evaluación de nivel de riesgo y de costos asociados al proyecto, depende del contexto del software y de su funcionalidad.
- Se define un criterio de aceptación, con colaboración del cliente, y en base a este criterio se determina cuanto testing se va a realizar. Puede ser un punto específico, un costo, un tiempo, etc.

TIPOS DE PRUEBA

- Smoke test (busca una falla catastrófica antes de arrancar).
    - Es la primera corrida de los test de sistema en la cual se verifica que las funcionalidades básicas de una aplicación o sistema funcionen correctamente antes de realizar pruebas más exhaustivas.
    - Se realiza antes de comenzar el ciclo 0. Nos ahorra empezar a testear formalmente si encuentra un error
- Testing Funcional (QUE hace el sistema):
    - Controla que el software se comporte de la misma manera que lo especificado en la documentación, cumpliendo con las funcionalidades y características definidas.
    - Basado en:
        - Basado en requerimientos: cuando se prueban requerimientos específicos, apunta a probar una funcionalidad sola (utilizan a los requisitos definidos en una ERS o los acuerdos que contienen las pruebas de usuario y los criterios de aceptación de una US para realizar las pruebas.)
        - Basado en proceso de negocio: cuando se prueba un proceso de negocio completo, es decir, se prueba todo el proceso. Por ejemplo, en una venta se prueba la búsqueda del artículo, la selección y facturación del mismo.
- Testing No Funcional (COMO lo hace el sistema):

    - Se basa en cómo trabaja el sistema haciendo foco en los requerimientos no funcionales.
    - Son las pruebas más complejas, por su gran dependencia al entorno del sistema, por lo que debe ser lo más parecido posible al del cliente.
    - Pruebas de testing, Carga, Stress, Usabilidad, Mantenimiento, Portabilidad, Faibilidad.
- Sanity Test (asegura que todos los test se corren para ver si las funcionalidades básicas están funcionando correctamente)
    - Se realiza después de una ronda de pruebas más exhaustivas para verificar si se han corregido los errores o problemas críticos identificados previamente.
    - El objetivo principal es asegurarse de que las correcciones realizadas no hayan introducido nuevos problemas y que las funcionalidades clave del software sigan funcionando correctamente después de las correcciones.

TDD / Test-Driven Development

- Es una práctica de desarrollo de software que se enfoca en escribir pruebas antes de desarrollar código. Se centra en el desarrollo funcional.
- Involucra dos prácticas:
    - Test First Development: primero se escriben las pruebas unitarias referentes a la característica de producto a implementar. Definidas las pruebas unitarias, se piensa en la codificación necesaria para que las pruebas se ejecuten con éxito.
    - Refactoring: consiste en reestructurar el código existente sin modificar el comportamiento que el mismo provee. Implica la mejora de aspectos funcionales del software, cuyo objetivo es mejorar la claridad del código y reducir su complejidad, con el fin de hacerlo más mantenible.

RED : test fails
GREEN: test passes
BLUE: refactor

Primero elijo el requerimiento que voy a probar, escribo la prueba, escribo la implementación de tal manera que logremos que el test pase. Lo importante es que el teste no falle (Green) una vez que logre que pase la prueba, hago refactorización, implemento el código de la mejor manera posible.

TRADICIONAL	TDD
Los desarrolladores diseñas las pruebas unitarias después de escribir el código	Los desarrolladores diseñas las pruebas unitarias antes de escribir el código
Aseguramiento y control de calidad más reactivo	Aseguramiento y control de calidad más proactivo

PASOS PARA REALIZAR UN TDD

- Escribir un test
    - Antes de escribir una línea de código tengo que pensar cómo voy a probar esa línea
    - Analizo los escenarios posibles y escribo un test que represente el comportamiento esperado del sistema antes de codear
- Hacer que el test compile
    - Una vez escrito el test, se agregan las clases, métodos o estructuras mínimas necesarias para que el código pueda compilar correctamente
    - En esta etapa todavía no importa si la funcionalidad funciona, sino que el entorno esté preparado para ejecutar la prueba.
- Hacer que el test falle (Red)

    - Se ejecuta el test y se verifica que falle, esto confirma que la funcionalidad aún no está implementada y que el test realmente está comprobando algo válido.
    - El estado “Red” representa justamente el fallo esperado de la prueba.
- Hacer un cambio para que pase el test
    - Se implementa el código mínimo necesario para que la prueba deje de fallar.
    - El objetivo no es crear una solución perfecta, sino una implementación simple que cumpla con lo que el test pide.
- Correr el test para que pase (Green)
    - Luego de realizar los cambios, se ejecutan nuevamente las pruebas para comprobar que ahora pasan correctamente.
    - El estado “Green” indica que la funcionalidad requerida ya cumple con las condiciones definidas en el test.
- Quitar duplicaciones (Refactor)
    - Se explica más abajo

Este ciclo red – green – refactor es el corazón de TDD

BENEFICIOS

- Obtenemos un código robusto, un código
    - Más predecible y limpio (permite saber cuándo el código está terminado)
    - Más tolerable al cambio, al tener escritas las pruebas es más fácil de entender
    - Más seguro
    - Aumenta la posibilidad de reducir la duplicación de código (si el test pasa el código ya existe)
    - Más barato de mantener, al ser más entendible
- Mayor velocidad de desarrollo y permite despliegue continuo
- Asegura cobertura de prueba, lo que conlleva a código de calidad
- Obliga a pensar en cómo usar el componente, por lo tanto, el diseño de las API
- Promueve el desarrollo de componentes con alta cohesión y bajo acoplamiento
- Si pienso primero en las pruebas reduzco errores

LEYES DE TDD

- No escribir una línea de código hasta que no hayas escrito antes un test case que falla
- No es necesario escribir más test de los necesarios para que falle. Normalmente con un test que falle es suficiente.
- No vas a escribir más código en Producción que el necesario para que el test pase.

REFACTORING

- Forma disciplinada de introducir cambios reduciendo la posibilidad de introducir defectos (básicamente mejoro y reorganizo el código sin modificar el comportamiento ya validado)
- Es una transformación del software que

    - Preserva la estructura externa
    - Mejora la estructura interna
- Son cambios que se hacen en el sistema que
    - No cambien el comportamiento observable (todas las pruebas aún pasan)
    - Eliminan la duplicación o complejidad innecesaria
    - Mejoran la calidad del softwar
    - Hacen un código más fácil de entender y de cambiar
    - Flexibiliza el código
- Es necesario porque
    - Evita el "deterioro del diseño"
    - Incrementar la legibilidad y la comprensibilidad (ya que lo ordena y simplifica)
    - Encuentra errores
    - Reduce el tiempo de depuración
    - Incorpora el aprendizaje que hacemos sobre la aplicación
    - Rehacer las cosas es fundamental en todo proceso creativo

TESTING EN AMBIENTES AGILES

Manifiesto del testing → Surge con el Manifiesto Ágil

- Valoramos el testing durante todo el proceso sobre las pruebas al final. Tradicionalmente el testing ha sido una fase que se ha producido al final del desarrollo. Sin embargo, en la agilidad el testing es una actividad que necesita darse junto con la codificación, la documentación y el resto de operaciones.

Si un tablero Kanban posee una columna de Testing, ten por seguro que el testing se está viendo como una fase y la actividad de los testers se está separando de la del resto del equipo. Como alternativa se recomienda una fase de Verificación donde una vez que el trabajo esté a DONE un compañero pueda revisarla.

- Valoramos la prevención de errores en lugar de encontrar errores. Debe hacerse lo antes posible, normalmente la mayoría de los errores se dan por asunciones sin aclarar en la toma de requisitos.
- Valoramos la comprensión de las pruebas sobre la verificación de la funcionalidad. Los testers deben ser los representantes del cliente en cada decisión del diseño, asegurándose que la feature satisfaga las verdaderas necesidades de estos y no solo las especificaciones.
- Valoramos construir el mejor sistema sobre romper el sistema. Tradicionalmente los Testers se han centrado en buscar la forma de romper el sistema, enfocándose en el Testing Negativo. Sin embargo, un tester se debe enfocar en el objetivo principal: crear un producto que aporte valor y cumpla su función como se espera.
- Valoramos la responsabilidad del equipo por la calidad sobre la responsabilidad del probador. Los testers por sí sólos no pueden mejorar la calidad. Su función es la de determinar el nivel de calidad e informar de ella a los interesados. Sólo puedes mejorar la calidad uniendo tus esfuerzos a los del equipo.

Principios del testing ágil

- el testing se mueve hacia adelante en el proyecto: no se realiza al final del proyecto, sino que se va haciendo de manera constante al finalizar cada iteración.
- testing no es una fase: se hace mientras se desarrolla el código

- todos hacen testing: no hay un rol especifico de tester, cualquiera dentro del proyecto puede testear.
- reducir la latencia del feedback
- las pruebas representan expectativas
- mantener el codigo limpio, corregir los defectos rapido: cuando hablamos del manifiesto hablamos de prevenir errores antes de solucionarlos
- reducir la sobrecarga de documentacion en las pruebas
- las pruebas son parte del "done": en el criterio de done se incluye la realización de pruebas.
- de probar al final a "conducido por pruebas"

Practicas del testing ágil

- testing unitario o de integración automatizado: todo lo que yo ejecuto de manera mecánica se automatiza. gano tiempo, ahorro recursos y los utilizo en otro contexto.
- testing regresivo a nivel de sistema automatizadas: las pruebas de regresión también podrían automatizarse.
- testing exploratorio: el testing exploratorio no se puede automatizar
- tdd (test driven development)
- atdd (desarrollo conducido por pruebas de aceptación): esta relacionado con tdd, pero tiene que ver con el desarrollo conducido por las pruebas de aceptación.
- control de versión de las pruebas con el código: junto con el código tengo asociados los unit tests y los system tests que tengo que correr y de esa manera es fácil plantear la automatización del testing

AUTOMATIZACIÓN

- Automatizar tiene varios beneficios y suma valor agregado
    - Rápida ejecución, Reusabilidad, Mayor cobertura, precisión y alcance, Reducción de costos, Feedback rápido
- Puedo automatizar
    - Pruebas unitarias, Pruebas de integración, Test de regresión
- Esto se logra mediante la integración de ciertas herramientas que permiten automatizar
- Lo hacemos gradualmente (empezando por los tests más críticos o repetitivos, automatizando primero pruebas simples y ampliando la cobertura iteración tras iteración)

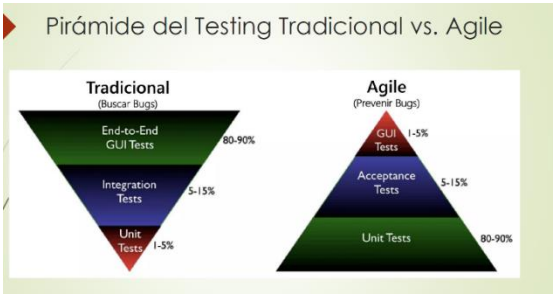
ROLES Y COMPETENCIAS DEL TESTER

- Tester en ambiente tradicional
    - El tester suele trabajar al final del desarrollo, cuando el sistema ya fue construido.
    - Busca encontrar errores de forma destructiva (se asume que existen errores)
    - Algunas tareas son
        - Diseñar y ejecutar casos de prueba, Detectar y reportar bugs, Validar y verificar, Documentar resultados.
- Tester en ambiente ágil
    - Es un rol, no una persona (todos son testers)

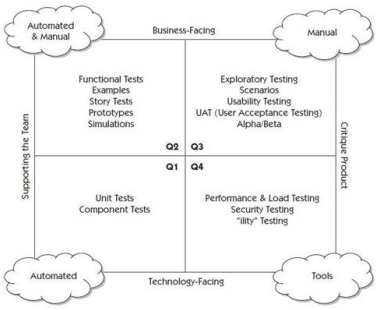
- Participa durante todo el Sprint y trabaja de manera colaborativa con Developers, Product Owner y System Team.
- Busca evitar que aparezcan errores
- Algunas tareas
  - Validación del “Done” de las historia, Registro y comunicación de bugs, Métricas de calidad, Aceptación de la historia con el PO, Preparación y mantenimiento de ambiente de pruebas, Casos y data de pruebas de las historias del sprint.

PIRAMIDE ÁGIL VS TRADICIONAL

- La diferencia principal está en dónde se concentra el esfuerzo de prueba y en el objetivo del testing.
- Tradicional
  - La mayor parte de las pruebas se realiza en la capa superior, mediante pruebas End-to-End o de interfaz gráfica (GUI).
  - Estas pruebas verifican el sistema completo, pero suelen ser más lentas, costosas y difíciles de mantener.
  - El testing ocurre al final del desarrollo (orientado a buscar errores al final del proceso)
  - La mayor parte de las pruebas son manuales y tienen poca automatización
- Ágil
  - Base → propone una base mucho más grande de pruebas unitarias / pruebas de componentes realizadas en etapas tempranas por los desarolladores (permiten validar pequeñas partes del código de manera rápida y automática, permitiendo detectar errores antes de que se propaguen)
  - Nivel intermedio → pruebas funcionales – reglas de negocio – capa de servicio
  - Parte superior → pocas pruebas de flujo de trabajo mediante interfaz gráfica, ya que son más lentas y menos prácticas de automatizar.
  - Buscamos prevenir bugs mediante automatización temprana, integración continua y retroalimentación rápida,
  - Las pruebas de aceptación y unitarias se automatizan (tratando de reducir al mínimo las pruebas manuales)



Cuadrantes del testing



Nos hablan de cuales son todos los tipos de testing disponibles, cual es el foco de cada uno, hacia donde apuntan y cuales son aquellos que pueden automatizarse y los que deben realizarse de manera manual.

Los cuadrantes 1 y 4 tienen que ver con aspectos tecnológicos del testing, y los 2 y 3 tiene más que ver con la parte del negocio del testing.

Específicamente el cuadrante 4 tiene que ver con aspectos no funcionales como tests de performance, seguridad, etc.

Los cuadrantes 1 y 2 tienen más relación con el soporte al equipo y los 3 y 4 están más orientados a lo que hacemos sobre el producto.